



LOMBA KOMPETENSI SISWA (LKS) SEKOLAH MENENGAH KEJURUAN TINGKAT PROVINSI JAWA BARAT TAHUN 2026

NASKAH SOAL

Tertutup

Bidang Lomba

CLOUD COMPUTING



**PEMERINTAH DAERAH PROVINSI JAWA BARAT
DINAS PENDIDIKAN**

Jalan Dr. Radjiman No. 6 Telp. (022) 4264813 Fax. (022) 4264

Website: diskdik.jabarprov.go.id

e-mail: diskdik@jabarprov.go.id/sekretariatdiskdikjabar@gmail.com

BANDUNG - 40171

Infrastructure Automation

Duration	Difficulty	Platform	Focus
4 Hours	Easy	AWS Academy	Infrastructure

1. PROJECT DESCRIPTION

1.1 Background & Context

Modern cloud-native applications demand more than just running containers — they require automated provisioning, multi-region resilience, real-time observability, and security by design. A development team that deploys manually, without infrastructure-as-code, without a CI/CD pipeline, and without cross-region monitoring cannot scale reliably or recover quickly in a production environment. The gap between 'it works on my machine' and 'it runs on AWS across two regions simultaneously' is precisely what this module is designed to close.

This module challenges participants to design and deploy a production-ready containerised web application on Amazon Web Services within three hours. The system is built on two Python Flask microservices: a Frontend service (port 3000) that renders a full User Management interface via Jinja2 HTML templates, and an API service (port 8080, with a Prometheus metrics endpoint on port 9100) that provides a complete REST CRUD interface backed by an Amazon RDS PostgreSQL database. Both services run on Amazon ECS Fargate inside a dedicated VPC in the us-east-1 (N. Virginia) region.

What distinguishes this project is its Inter-Region VPC Peering component. Participants must create a second VPC in us-west-2 (Oregon), deploy a monitoring stack (Prometheus + Grafana) on ECS Fargate inside that VPC, and connect the two VPCs using an inter-region peering connection named pcx-lks-2026. An EC2 bastion instance in Oregon must successfully ping the private IP of an ECS container in Virginia via SSM Run Command — proving that peering, routing, and Security Groups are all configured correctly. The entire infrastructure lifecycle is automated through GitHub Actions and Terraform, so a single git push is enough to provision or update the entire platform from end to end.

1.2 Task List

Complete all eight tasks in the order listed. Each task references the relevant section of this document for detailed specifications and requirements.

1. Log in to the AWS Management Console using your AWS Academy Learner Lab credentials. Confirm the active region is us-east-1. Prepare your GitHub repository and configure all required Secrets. See Section 8.
2. The Terraform modules are provided in the repository but contain intentional bugs. Find and fix all bugs, then run terraform apply to provision the infrastructure. See Section 3.
3. Confirm that three ECR repositories have been created. Review the Flask Dockerfiles and verify that both images build correctly by running docker compose up -d. See Section 5.
4. Push code to the main branch and verify that all four GitHub Actions jobs complete successfully. Confirm Docker images are available in ECR with the correct tags. See Section 8.
5. Verify that the ECS cluster lks-ecs-cluster is ACTIVE. Confirm lks-fe-service and lks-api-service are ACTIVE with running tasks and healthy Target Groups. See Section 5.

6. Access the application through the ALB URL. Test the full CRUD cycle: create, read, update, and delete a user. See Section 6.
7. Verify lks-monitoring-vpc (10.1.0.0/16) exists in Oregon. Confirm VPC Peering connection pcx-lks-2026 is active and that route tables in both regions contain the correct cross-region routes. See Section 4.
8. Verify that EC2 bastion lks-bastion-oregon is running. From the Oregon bastion, ping the private IP of a running ECS task in Virginia via SSM Run Command. A successful ping proves that inter-region peering is fully operational. See Section 7.

1.3 Technical Specifications

The table below summarizes all technical specifications that must be met in the implementation of this module. These values are what the automated judge verifies against your live AWS resources.

Parameter	Required Value
Primary Region	us-east-1 (N. Virginia) Application Stack
Secondary Region	us-west-2 (Oregon) Monitoring & VPC Peering
Resource Prefix	All AWS resources must be prefixed with lks-
Application Stack	Python 3.13 + Flask 3.1 (API & Frontend)
Infrastructure	Terraform — VPC, Security Groups, ALB, RDS, DynamoDB, S3, SSM
CI/CD	GitHub Actions — 4-job pipeline: install → build_ecr → upload_s3 → deploy
Monitoring	Prometheus (scrape port 9100) + Grafana — ECS Fargate in Oregon
VPC Peering	Inter-region pcx-lks-2026 — always encrypted, DNS not supported
Bastion Verification	EC2 t3.micro in Oregon — SSM Run Command ping to Virginia ECS
Repository	https://github.com/cloudtechlks/infraautomation.git

1.4 References

1. Virtual Private Cloud - <https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html>
2. Elastic Load Balancing - <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>
3. Amazon ECS (Elastic Container Service) - <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html>
4. Amazon ECS Fargate Launch Type - https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html
5. Amazon ECR (Elastic Container Registry) - <https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html>
6. Amazon EC2 (Elastic Compute Cloud) - <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html>
7. Amazon RDS for PostgreSQL - https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_PostgreSQL.html
8. Amazon DynamoDB - <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>
9. Amazon S3 - <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>

10. Amazon SQS - <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html>
11. AWS SSM Parameter Store - <https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html>
12. AWS SSM Session Manager - <https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html>
13. Terraform AWS Provider - <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>
14. GitHub Actions Documentation - <https://docs.github.com/en/actions>
15. Docker - <https://docs.docker.com/get-started/overview/>
16. Docker Compose - <https://docs.docker.com/compose/>

1.5 SYSTEM ARCHITECTURE

1.5.1 Architecture Overview

The platform is organized around two VPCs in separate AWS regions that communicate through an encrypted inter-region VPC peering connection. The primary stack in Virginia serves end-user traffic; the secondary stack in Oregon provides observability and acts as the peering verification endpoint. All cross-region traffic flows over private IP addresses — no traffic crosses the public internet after the initial ALB ingress.

Virginia (us-east-1) Application Layer. An Application Load Balancer (ALB) accepts public HTTP traffic and routes it to two ECS Fargate services in private subnets based on URL path: the Frontend service (`/`) and the API service (`/api/`). The API service exposes a secondary metrics endpoint on port 9100 in Prometheus text format, which is scraped by the Prometheus instance in Oregon via the peering connection. RDS PostgreSQL 15 in an isolated subnet provides relational storage; DynamoDB and SQS handle sessions and asynchronous event publishing. All sensitive parameters (RDS endpoint, credentials) are stored in SSM Parameter Store and retrieved by ECS containers at startup via environment variables injected into the Task Definition.

Oregon (us-west-2) Monitoring & Verification Layer. A second VPC with no Internet Gateway hosts two ECS Fargate services: Prometheus (which scrapes the Virginia API metrics endpoint at `10.0.3.x:9100` via the peering route) and Grafana (which provides dashboards accessible through a dedicated monitoring ALB). An EC2 `t3.micro` bastion instance in the same private subnet is used exclusively for peering verification — SSM Run Command executes a ping to the Virginia ECS API task IP, confirming bidirectional L3 connectivity across the peering connection.

1.6 Service Map

The following table maps every AWS service used in this module to its role in the platform, helping you understand why each component exists before you build it.

Service	Layer	Role in the Platform
lks-alb	Ingress	Internet-facing ALB — routes <code>/</code> to frontend, <code>/api/</code> to API
Amazon ECS Fargate	Compute	Serverless container execution — no EC2 instances to manage
Amazon ECR	Registry	Docker image registry — lks-fe-app, lks-api-app, lks-monitoring
Amazon RDS PostgreSQL	Storage	Relational DB in isolated subnet — CRUD persistence
Amazon DynamoDB	Storage	lks-sessions — PAY_PER_REQUEST, session store with TTL
Amazon SQS	Messaging	lks-event-queue + DLQ — async user event publishing
AWS SSM Parameter Store	Config	Secure storage for DB endpoint and credentials
Terraform	IaC	Declarative provisioning of all networking and supporting infra
GitHub Actions	CI/CD	4-job pipeline — install → build ECR → upload S3 → apply

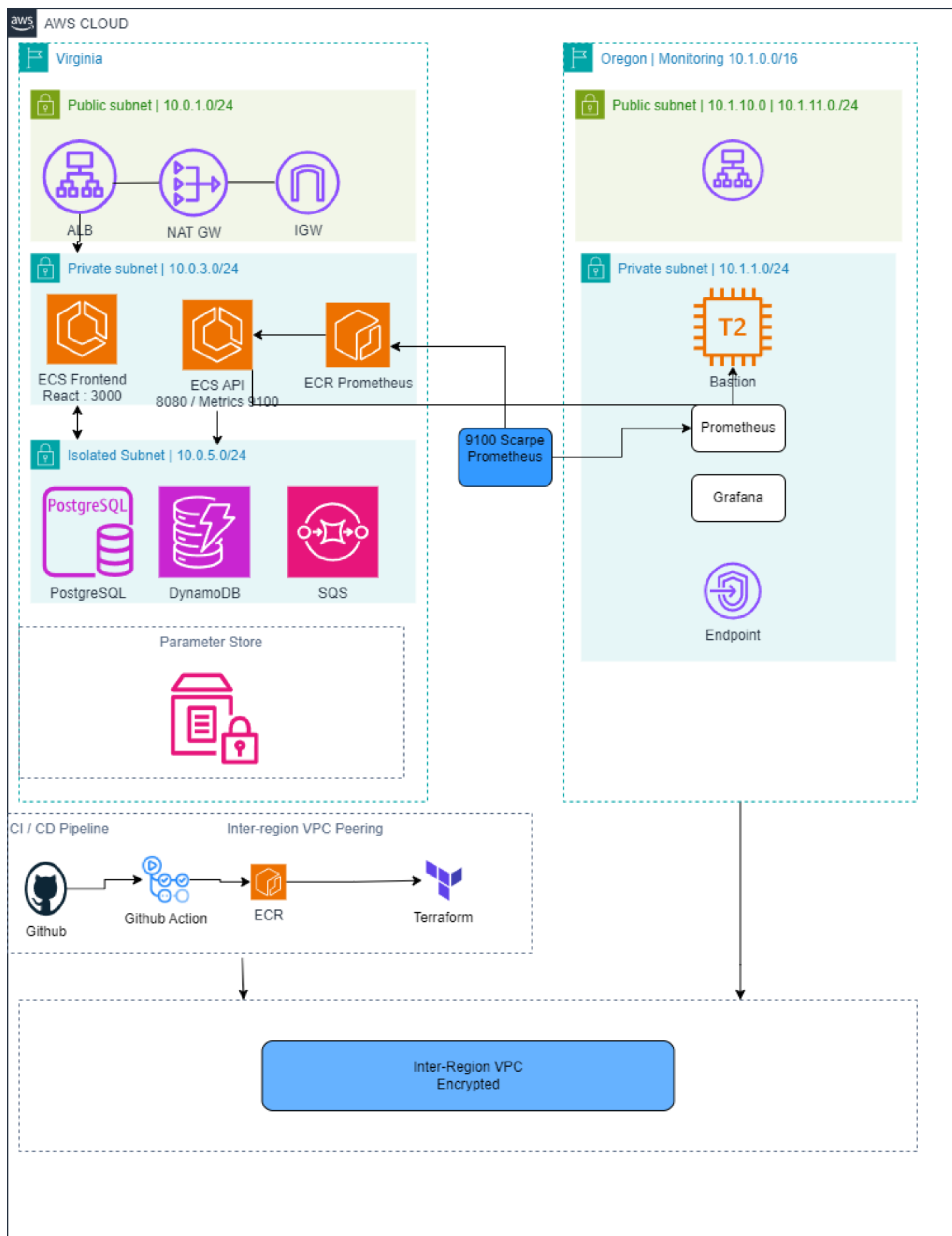
Prometheus (Oregon)	Monitoring	Scrapes API metrics endpoint via inter-region VPC peering
Grafana (Oregon)	Monitoring	Dashboards — exposed via monitoring ALB
VPC Peering pcx-lks-2026	Networking	Private inter-region connectivity Virginia ↔ Oregon
EC2 Bastion (Oregon)	Verification	SSM-connected instance — runs ping to Virginia ECS IP

1.7 Data Flow — End to End

Understanding the data flow helps you debug issues at each hop. A user request from the internet hits the ALB on port 80 in a public subnet in us-east-1. The ALB evaluates the path: requests to `/api/*` are forwarded to `lks-api-service` on port 8080, while all other requests go to `lks-fe-service` on port 3000. The API service writes user records to RDS PostgreSQL, stores session tokens in DynamoDB, and publishes events to SQS. A separate thread inside the API container listens on port 9100 and serves Prometheus-format metrics for every request it handles.

In the Oregon region, Prometheus sends HTTP scrape requests to `10.0.3.x:9100` — the private IP of the ECS API task in Virginia. These requests travel through the `pcx-lks-2026` peering connection, traverse the route table entries you must configure, and are admitted by the `lks-sg-ecs` Security Group rule that allows TCP 9100 from `10.1.0.0/16`. Grafana then reads from the local Prometheus instance and renders dashboards. The EC2 bastion in Oregon uses the same peering route to send ICMP ping packets to the same Virginia ECS IP, providing a simple, observable proof that the entire peering path is functional.

1.8 Architecture Diagram



2. NETWORKING — VPC & TERRAFORM INFRASTRUCTURE

2.1 What is Terraform and Why Use It?

Terraform is an open-source Infrastructure as Code (IaC) tool developed by HashiCorp. Instead of clicking through the AWS Console to create resources, you describe the desired state of your infrastructure in .tf configuration files. Terraform then calculates what needs to be created, modified, or destroyed and applies those changes via the AWS API. This approach makes infrastructure repeatable, version-controlled, and auditable. A push to your GitHub repository can trigger the entire stack to be rebuilt from scratch in minutes.

In this module, Terraform manages all core networking and supporting infrastructure. You write all .tf module files yourself based on the specifications below. The repository provides a modules/ folder with README files listing the resources that need to be

created. All AWS resource names must begin with lks-. Use LabRole for all IAM role references — do not attempt to create new IAM roles in AWS Academy Learner Lab.

The Terraform modules for this project have already been written and are available in the terraform/modules/ directory of your repository. However, the modules contain intentional bugs that will prevent the infrastructure from being provisioned correctly.

Your task is to read through the module files, identify all bugs, and fix them before running terraform apply.

Use the specifications in Sections 3.2 through 3.6 as your reference — every bug directly contradicts at least one requirement stated in this document.

Do not modify main.tf, variables.tf, or backend.tf in the root terraform directory. All bugs are located inside the terraform/modules/ folder.

2.2 Application VPC — lks-vpc (us-east-1)

A Virtual Private Cloud (VPC) is a logically isolated network within AWS. Think of it as your own private data centre inside the AWS cloud — you control the IP address range, subnet layout, routing, and firewall rules. The application VPC uses the CIDR block 10.0.0.0/16, which provides 65,536 IP addresses divided across six subnets in two Availability Zones for high availability.

Create lks-vpc with CIDR 10.0.0.0/16, DNS support and DNS hostnames enabled. Attach one Internet Gateway (for outbound internet access from public subnets) and provision one NAT Gateway in the first public subnet (so private-subnet containers can reach ECR and other AWS services without being directly internet-accessible). Create three route tables — public (routes to IGW), private (routes to NAT), and isolated (no internet route at all).

Subnet Name	Type	CIDR	Availability Zone
lks-public-subnet-1a	Public	10.0.1.0/24	us-east-1a
lks-public-subnet-1b	Public	10.0.2.0/24	us-east-1b
lks-private-subnet-1a	Private	10.0.3.0/24	us-east-1a
lks-private-subnet-1b	Private	10.0.4.0/24	us-east-1b
lks-isolated-subnet-1a	Isolated	10.0.5.0/24	us-east-1a
lks-isolated-subnet-1b	Isolated	10.0.6.0/24	us-east-1b

2.3 Security Groups

A Security Group acts as a virtual firewall that controls inbound and outbound traffic for AWS resources. Unlike a traditional firewall that operates at the network perimeter, Security Groups are attached directly to individual resources (EC2 instances, ECS tasks, RDS databases) and evaluate rules for every packet. You must create three Security Groups with the exact inbound rules shown below — the judge verifies both that the groups exist and that the required rules are present.

Security Group	Port / Protocol	Source	Purpose
lks-sg-alb	80, 443 TCP	0.0.0.0/0	Allow public web traffic to reach the ALB
lks-sg-ecs	3000, 8080 TCP	lks-sg-alb	Allow ALB to forward to ECS containers

lks-sg-ecs	9100 TCP	10.1.0.0/16	Allow Prometheus in Oregon to scrape metrics via peering
lks-sg-ecs	ICMP all	10.1.0.0/16	Allow ping from EC2 bastion in Oregon via peering
lks-sg-db	5432 TCP	lks-sg-ecs	Allow ECS containers to reach RDS PostgreSQL

2.4 Application Load Balancer

An Application Load Balancer (ALB) operates at Layer 7 (HTTP/HTTPS) and can route requests to different backend targets based on URL path, host header, or query string. This is how one ALB serves both the frontend and the API — requests to `/api/*` go to the API target group, while all other paths go to the frontend target group. This eliminates the need for two separate load balancers, and a single DNS name serves the entire application.

Target Group	Port	Health Check Path	Listener Rule Priority
lks-tg-api	8080	/api/health	Priority 1 (evaluated first)
lks-tg-fe	3000	/health	Default (catch-all)

Both target groups must use `target_type = 'ip'`. This is mandatory for ECS Fargate tasks, which do not have a persistent EC2 instance — the ALB registers the task's private IP address directly. When a task is stopped and replaced, ECS automatically updates the target group registration.

2.5 Database & Supporting Resources (Isolated Subnets)

All database resources are placed in isolated subnets with no route to the internet, not even through a NAT Gateway. This is the most secure placement for data stores: even if the ECS containers were compromised, an attacker could not reach the database directly from the internet. ECS containers can still reach the database because they are in private subnets that share the VPC's internal routing.

Resource	Name / Identifier	Key Configuration
RDS PostgreSQL	lks-rds-postgres	PostgreSQL 15, db.t3.micro, 20 GB gp2, DB: lksdb, user: lksadmin / LKSSecure2026!, not publicly accessible
DynamoDB	lks-sessions	PAY_PER_REQUEST billing, TTL attribute: expiresAt
SQS Queue	lks-event-queue	Standard queue with DLQ lks-dlq, maxReceiveCount: 3
SSM Parameter	/lks/app/db_host	String — RDS endpoint hostname
SSM Parameter	/lks/app/db_password	SecureString — RDS master password
SSM Parameter	/lks/app/sqs_url	String — SQS queue URL
SSM Parameter	/lks/app/dynamodb_table	String — DynamoDB table name

2.6 S3 Buckets

Two S3 buckets are provisioned by the `modules/s3` Terraform module. The `tfstate` bucket stores deployment metadata uploaded by the CI/CD pipeline. The `assets` bucket stores application files. Both buckets must have versioning enabled and a

lifecycle policy configured. Run terraform apply before pushing to GitHub — the pipeline will fail if the tfstate bucket does not exist.

Resources	Name / Identifier	Key Configuration
S3 Bucket	lks-tfstate-[name]-[suffix]	Versioning enabled, lifecycle: transition to Standard-IA after 30 days, delete after 90 days
S3 Bucket	lks-app-assets-[name]-2026	Versioning enabled, lifecycle: transition to Standard-IA after 30 days, delete after 365 days

3. MONITORING VPC & INTER-REGION VPC PEERING

3.1 What is VPC Peering?

VPC Peering is a networking connection between two VPCs that enables routing traffic between them using private IPv4 or IPv6 addresses — as if they were part of the same network. Inter-region VPC peering extends this across AWS regions. Traffic traverses Amazon's private global network backbone, not the public internet, and is always encrypted in transit. This provides low latency, high bandwidth, and strong security guarantees without requiring a VPN tunnel or Direct Connect circuit.

There are three things you must configure for peering to work. First, create the peering connection itself (one VPC is the requester, the other is the acceptor). Second, accept the connection on the acceptor side — for inter-region connections this must be done explicitly and cannot be auto-accepted. Third, add routes to both VPCs' route tables pointing each other's CIDR at the peering connection ID. Without all three steps, traffic will not flow even if the connection shows as Active.

3.2 Monitoring VPC — lks-monitoring-vpc (us-west-2)

The monitoring VPC is created inside modules/monitoring_vpc/ using providers = { aws = aws.oregon }. This VPC contains only two private subnets and has no Internet Gateway. All connectivity to AWS services (ECR image pulls, CloudWatch log delivery, SSM agent communication) is handled through VPC Interface Endpoints — private tunnels into AWS service APIs that never leave the Amazon network.

Subnet Name	CIDR	Availability Zone
lks-monitoring-private-1a	10.1.1.0/24	us-west-2a
lks-monitoring-private-1b	10.1.2.0/24	us-west-2b

The following VPC Interface Endpoints must be created in the Oregon VPC. Each endpoint creates an Elastic Network Interface (ENI) in your subnet and a private DNS record so that AWS SDK calls resolve to the endpoint rather than the public AWS IP. Without these, Fargate tasks in a VPC with no internet access cannot pull images, write logs, or communicate with ECS control plane.

- ssm, ssmmessages, ec2messages — required for SSM Run Command to reach the EC2 bastion
- logs — required for the awslogs CloudWatch driver used by Fargate tasks
- ecr.api, ecr.dkr — required for Fargate to pull images from ECR
- ecs, ecs-agent, ecs-telemetry — required for ECS control plane communication
- monitoring — required for Container Insights metrics

- s3 (Gateway type, free) required for ECR image layer downloads stored in S3

3.3 Peering Connection — pcx-lks-2026

Create the peering connection inside modules/vpc_peering/ using both the Virginia and Oregon providers simultaneously. Set `peer_region = 'us-west-2'` on the `aws_vpc_peering_connection` resource. Use `aws_vpc_peering_connection_accepter` with `provider = aws.oregon` — the `auto_accept` attribute is not supported for inter-region connections and must be omitted.

Property	Value
Connection Name	pcx-lks-2026
Requester VPC	lks-vpc — 10.0.0.0/16 (us-east-1)
Accepter VPC	lks-monitoring-vpc — 10.1.0.0/16 (us-west-2)
Encryption	Automatic — always on for inter-region peering
DNS Resolution	NOT supported — use private IP addresses only
Verification Proof	EC2 bastion in Oregon pings private IP of ECS task in Virginia

Route table entries required in both regions:

Route Table	Destination CIDR	Target
lks-private-rt (us-east-1)	10.1.0.0/16	pcx-lks-2026
lks-monitoring-rt (us-west-2)	10.0.0.0/16	pcx-lks-2026

3.4 Monitoring Stack — ECS Fargate (us-west-2)

Prometheus is an open-source monitoring system that collects metrics by periodically sending HTTP GET requests to endpoints that expose data in a standardized text format. The Flask API container runs a separate mini-HTTP server on port 9100 that serves these metrics — request counts, latency histograms, error rates, and custom business metrics. Prometheus pulls and stores this data; Grafana then connects to Prometheus as a data source and renders interactive dashboards with graphs, gauges, and alerts.

Both monitoring services run in `lks-monitoring-cluster` in the private subnet. Docker images are pushed from the CI/CD pipeline to ECR in us-west-2 (repository `lks-monitoring`) and pulled by Fargate through the ECR VPC endpoint. Prometheus is configured to scrape `10.0.3.x:9100` — the private IP of the ECS API task in Virginia — through the peering route.

Service	Image	Port	ALB Listener
lks-grafana-service	ECR: lks-grafana	3000	lks-monitoring-alb :80 → port 3000
lks-prometheus-service	ECR: lks-prometheus	9090	lks-monitoring-alb :8080 → port 9090

4. ECR, DOCKER & ECS FARGATE

4.1 What is Amazon ECR?

Amazon Elastic Container Registry (ECR) is a fully managed Docker-compatible container image registry hosted within AWS. Unlike Docker Hub, ECR is private by default and integrates natively with IAM for access control and with ECS for automatic credential management. When an ECS Fargate task starts, it authenticates to ECR using its Task Execution Role (LabRole in this module), pulls the specified image, and starts the container — all without any manual docker login step.

Repository Name	Region	Used By
lks-fe-app	us-east-1	ECS Frontend Service — Flask, port 3000
lks-api-app	us-east-1	ECS API Service — Flask, port 8080 + 9100
lks-monitoring	us-west-2	Monitoring stack — Grafana + Prometheus images

4.2 Docker Specifications

Docker is a platform for packaging applications and their dependencies into lightweight, portable containers. A Dockerfile describes the steps to build a container image: starting from a base image, copying application code, installing dependencies, and specifying how to start the application. Both microservices use multi-stage builds based on python:3.13-slim to keep image sizes small. All containers run as a non-root user (appuser) as a security best practice. Configuration values that differ between environments (API URL, database host, passwords) are never baked into the image — they are injected at runtime through ECS Task Definition environment variables.

Service	Base Image	Port	Key Environment Variables
Frontend	python:3.13-slim	3000	PORT, API_URL, GRAFANA_URL, PROMETHEUS_URL
API	python:3.13-slim	8080 + 9100	DB_HOST, DB_PASSWORD, DB_SSL=false, PYTHONUNBUFFERED=1

4.3 Flask Application Structure

Flask is a lightweight Python web framework. The API service uses Flask to expose REST endpoints backed by PostgreSQL; the Frontend service uses Flask with Jinja2 templates to render HTML. The two services are deliberately kept separate — the Frontend calls the API over the internal ALB path /api/*, not directly to the container IP. This mirrors a real-world microservice architecture where services communicate through defined interfaces.

File	Purpose
frontend/app.py	Flask app — renders Jinja2 templates, injects API_URL, GRAFANA_URL, PROMETHEUS_URL from ECS env
frontend/templates/index.html	User Management UI — full CRUD in vanilla JS, dark theme, status bar for API/DB/Prometheus/Grafana
api/app.py	/api/health always 200, background DB init thread, /api/status proxy-check for Prometheus+Grafana
api/db.py	psycopg2 connection pool — 10x retry with exponential backoff; CRUD routes return 503 while connecting

api/routes/users.py	Full CRUD: GET/POST /api/users, PUT/DELETE /api/users/:id
api/metrics.py	Separate Flask app on port 9100 — serves Prometheus text-format metrics
api/sqs.py	boto3 SQS publisher — publishes user events to lks-event-queue

4.4 REST API Endpoints

A REST API (Representational State Transfer Application Programming Interface) is a standard way for software systems to communicate over HTTP. Each endpoint is identified by a URL path and an HTTP method (GET, POST, PUT, DELETE). The API service implements a complete CRUD interface for user management. The judge tests these endpoints directly through the ALB — make sure they work before submission.

Method	Endpoint	Description	Response
GET	/api/health	Health check — always 200, reports DB status	{"status":"ok","db":"..."}
GET	/api/status	Server-side proxy check for Prometheus + Grafana	{"prometheus":{"..."},"grafana":{"..."}}
GET	/api/users	List all users	[{"id":1,"name":"..."}]
POST	/api/users	Create a new user	201 + user object or 409 if duplicate
PUT	/api/users/{id}	Update an existing user	200 + user or 404
DELETE	/api/users/{id}	Delete a user	204 No Content or 404

4.5 ECS Service Specifications

Amazon ECS (Elastic Container Service) is a fully managed container orchestration service. With the Fargate launch type, you specify how much CPU and memory a task needs, provide a Docker image, and ECS handles everything else — finding a host, starting the container, monitoring its health, and replacing it if it crashes. You do not manage any EC2 instances. A Service maintains a desired count of running tasks and automatically registers them with the Target Group so the ALB can forward traffic to healthy containers.

Service	Image	CPU	Memory	Min / Max Tasks
lks-fe-service	lks-fe-app:latest	256	512 MB	1 / 3
lks-api-service	lks-api-app:latest	512	1024 MB	1 / 3

5. APPLICATION VERIFICATION

5.1 User Management Interface

Once the ALB is provisioned and ECS services are running, open the ALB DNS name in your browser. The dark-themed User Management interface should load and display an empty user table. The status bar across the top of the page provides real-time indicators of every downstream dependency. All four indicators must be green before submission.

Status Indicator	What It Confirms
API: Online	ALB routing → ECS API service is functional
Database: Connected	API container can reach RDS PostgreSQL in the isolated subnet

Prometheus: Online	/api/status proxy successfully reached the Oregon Prometheus
Grafana: Online	/api/status proxy successfully reached the Oregon Grafana

5.2 CRUD Verification Sequence

Test the full CRUD lifecycle in order. The judge performs these same steps automatically against the live ALB endpoint during scoring. Ensure each step succeeds before moving to the next — a failure at the Create step means Read, Update, and Delete cannot be meaningfully tested.

Step	Action	Expected Outcome
1 — CREATE	Fill in name, email, institution, position, phone — click Save	User appears in the table; API returns HTTP 201
2 — READ	Refresh the page or click the reload button	User persists in the table after page reload
3 — UPDATE	Click Edit on the user, change a field, click Save	Updated data appears immediately in the table
4 — DELETE	Click Delete on the user and confirm the dialog	User is removed from the table; API returns HTTP 204

6. EC2 BASTION & PEERING VERIFICATION

6.1 What is an EC2 Bastion and Why SSM?

A bastion host (also called a jump server) is a special-purpose server used to provide controlled access to a private network. Traditionally, bastions were accessed via SSH over a public IP address — a significant security risk. AWS Systems Manager (SSM) Session Manager eliminates the need for open inbound SSH ports entirely. Instead, the SSM agent running on the EC2 instance maintains an outbound connection to the SSM service (via VPC Interface Endpoints in this module). You send commands through this channel — no inbound port 22 is ever opened, and no SSH key pair is required.

The lks-bastion-oregon instance is an EC2 t3.micro running in a private subnet of lks-monitoring-vpc. It has no public IP address. All access is through AWS Systems Manager Run Command via the VPC Interface Endpoints. The instance must be tagged with Name = lks-bastion-oregon and associated with LabRole as its instance profile.

6.2 Required SSM VPC Endpoints (Oregon)

Three VPC Interface Endpoints must exist in the Oregon private subnet to enable SSM connectivity to the bastion. Without these, SSM Run Command cannot reach the instance because the VPC has no internet access for the SSM agent to communicate over.

- com.amazonaws.us-west-2.ssm — SSM agent communication channel
- com.amazonaws.us-west-2.ssmmessages — SSM Session Manager tunnelling
- com.amazonaws.us-west-2.ec2messages — SSM Run Command delivery

6.3 Ping Verification — Proof of Peering

The final verification step executes a ping from the Oregon bastion to the private IP address of a running ECS API task in Virginia. ICMP ping is a Layer 3 (network layer) protocol — a successful ping proves that IP routing through the peering connection is working correctly and that Security Group rules on both ends permit the traffic. This is the most reliable end-to-end proof of peering because it exercises every component in the path: route table entries, peering connection, Security Group rules, and the container network configuration.

Step 1: Find the private IP of the running ECS API task in Virginia:

```
aws ecs describe-tasks --cluster lks-ecs-cluster \
--tasks $(aws ecs list-tasks --cluster lks-ecs-cluster \
--service-name lks-api-service \
--query 'taskArns[0]' --output text) \
--query 'tasks[0].attachments[0].details[?name==`privateIPv4Address`].value' \
--output text --region us-east-1
```

Step 2: Send the ping via SSM Run Command targeting the bastion in Oregon:

```
aws ssm send-command --region us-west-2 \
--instance-ids <BASTION_INSTANCE_ID> \
--document-name "AWS-RunShellScript" \
--parameters '{"commands":["ping -c 4 <ECS_PRIVATE_IP>"]}' \
--query 'Command.CommandId' --output text
```

Expected output: 4 packets transmitted, 4 received, 0% packet loss. This confirms that the peering connection pcx-lks-2026 is fully operational, routes are correct in both VPCs, and Security Groups permit ICMP traffic from 10.1.0.0/16.

7. CI/CD PIPELINE — GITHUB ACTIONS

7.1 What is CI/CD and Why Does It Matter?

Continuous Integration (CI) is the practice of automatically building and testing code every time a change is pushed to the repository. Continuous Deployment (CD) extends this by automatically deploying the tested code to the target environment. Together, CI/CD transforms deployment from a manual, error-prone ceremony into a reliable, repeatable automated process. GitHub Actions is GitHub's built-in CI/CD engine — it runs workflows defined as YAML files in the `.github/workflows/` directory whenever specified events occur, such as a push to the main branch.

In this module, a single push to main triggers a four-job workflow that builds Docker images, pushes them to ECR, uploads deployment metadata to S3, and runs terraform apply to update the ECS Task Definitions with the new image tags. The entire cycle — from git push to running containers — should complete in under ten minutes.

7.2 Four-Job Pipeline

Job Name	Actions Performed
1 — install	Install Python dependencies for api + frontend; run smoke-test imports to catch syntax errors early
2 — build_and_push_ecr	Authenticate to ECR, build Docker images for Flask services, push with commit SHA as image tag
3 — upload_to_s3	Upload deployment metadata JSON (commit SHA, image URIs, timestamp) to S3 bucket lks-tfstate-*

4 — deploy	Run terraform init → validate → plan -var image_tag=<SHA> → apply to update ECS Task Definitions
------------	--

Each job depends on the success of the previous one (needs: [...] in the YAML). If the Docker build fails, Terraform never runs. This prevents deploying broken images to production. The commit SHA is used as the image tag throughout — this makes every deployment traceable back to the exact code commit that produced it.

7.3 Required GitHub Secrets

GitHub Secrets are encrypted key-value pairs stored in the repository settings that are injected as environment variables into workflow runs. They are never printed in logs or visible to workflow code as plain text after the initial set. AWS Academy credentials must be stored here — never commit them to your repository. Configure all secrets below in Settings → Secrets and variables → Actions → New repository secret.

Secret Name	Where to Find the Value
AWS_ACCESS_KEY_ID	AWS Academy → AWS Details tab → AWS CLI section
AWS_SECRET_ACCESS_KEY	AWS Academy → AWS Details tab → AWS CLI section
AWS_SESSION_TOKEN	AWS Academy → AWS Details tab — refresh at each session start
AWS_REGION	us-east-1
MONITORING_REGION	us-west-2
AWS_ACCOUNT_ID	Your 12-digit AWS account ID
ECR_REGISTRY	[account-id].dkr.ecr.us-east-1.amazonaws.com
TF_STATE_BUCKET	lks-tfstate-[yourname]-2026 (must already exist in S3)
STUDENT_NAME	Your name — used for resource naming

— GOOD LUCK —